

Standard Operating Procedure (SOP)

Development Team

Date : 01-Jan-2026

1. Introduction

This Standard Operating Procedure (SOP) outlines the professional standards, development guidelines, workflow structure, quality expectations, and responsibilities for Developers at BarodaWeb. All developers must strictly adhere to this SOP to ensure consistency, security, performance, and high-quality project delivery.

2. Objective of the SOP

- Establish a uniform development process for all MERN/Next.js projects.
- Standardize admin panel, server panel, and front-end behavior.
- Maintain high-quality coding, optimized APIs, and secure authentication.
- Ensure proper communication and accountability.
- Protect project quality through formal warnings for repeated mistakes.

3. Roles & Responsibilities

MERN / Next.js Developer Responsibilities

- Develop full-stack applications using MongoDB, Express.js, React.js/Next.js, and Node.js.
- Follow SEO and performance guidelines. (For Next.js Application)
- Ensure mobile responsiveness and front-end accuracy as per design.
- Maintain clean, readable, tested, and optimized code.
- Follow strict image, video, file upload standards.
- Coordinate with Designers, QA, PM, and Support Team.
- Manage deployments on BarodaWeb hosting infrastructure.
- Ensure zero unauthorized design or functional modifications.
- All code must be available on the Baroda web Github account, Any personal account uses strict action.

4. Technology Stack

Component	Technology	Remarks
Front-End (Main Website)	Next.js & React.js	<u>Next.js</u> → mandatory for SEO; SSR required. If SEO is not needed → Develop the application in <u>React.js</u> . Rupesh Sir : Approved → React Js for both SEO & Normal
Front-End & Backend Language	JavaScript, TypeScript	TypeScript only when required.
Backend (Admin Panel)	Node.js + React.js	For all admin panel and server-side modules.
Database	MongoDB	PostgreSQL only on management/client approval.
State Management	Redux Toolkit, RTK Query / ContextApi	Standard for APIs and store handling.
API	REST APIs	GraphQL only with approval from Rupesh Sir & Shimyon.
Data Fetching	Axios	Default.
CSS	React-Bootstrap, Material UI, Bootstrap	Tailwind only for custom-from-scratch UI.
Authentication	JWT	OAuth2 for large-scale or specific client requirements.
Environment Variable	.env file	All environments must be set on a .env file, Don't hardcode any global value.
Testing	Jest, Selenium	Unit + Integration Testing.
CI/CD	GitHub Actions / Action Runner	Action Runner requires Management approval.
Hosting + SSL	BarodaWeb Infrastructure	Mandatory.

5. Admin & Server Panel

5.1 Form Handling (Add / Edit)

- Show loader: “This process might take a few seconds...”
- After successful operation, show toast: “**Data submitted/updated successfully.**”
- All required fields must display an asterisk (*).
- All mandatory fields must include validation messages.
- Duplicate Entry or Multiple Entry not created.

5.2 Image Upload

Required Display Information:

- Max size & dimensions must be mentioned.
- Allowed file types: *.jpg, .jpeg
- Accept images up to **2MB**.

Standard Image Sizes:

- **Products / Services / Gallery:** 100KB–200KB
- **Banner Images:** 250KB–350KB
- **Client/Logo Images:** 20KB–50KB

Processing Rules :

1. Image Handling

- Auto-crop or fixed-size enforcement should be applied to all image uploads.
- If no image is uploaded, a “**No Image**” placeholder should be displayed.
- The backend should convert all images to **WebP (preferred)** or **AVIF** format.

2. Project Closure Guidelines

- When the project is nearing closure, only **project-related images** should be added.
- Before going live, please ensure that **all images and uploaded data are properly visible and verified on the live site.**

3. Naming Convention

- Use meaningful, real names related to the **service, product, page, image, PDF, or video**.
- URLs should follow a clear and readable structure, for example:
`www.domainname.com/services/web-application`
- File names should align with the page or service title, such as:
`web-application.webp` or `webapplication.webp`
- Avoid using **underscores (_), spaces, equals signs (=), or any special symbols** in URLs or file names for better performance and consistency.

4. Consistency

- The same naming convention should be followed at the time of uploading **images, PDFs, videos**, and for any **hardcoded assets**.
- File names should always reflect the **page or title name** for better organization and SEO consistency.

5.3 Video Upload

- Mention max size & dimensions.
- Allowed types: `.m4v`, `.mp4`
- By default, **only the YouTube link** option must be shown.
- Upload video file option only if client requests, also check with hosting space.

5.4 File Upload

- Allowed types: `.pdf`, `.doc`
- Size limit must be displayed.
- If exceeded → show error alert.

5.5 Delete Function

- Show confirmation popup: *“Are you sure you want to delete this item?”*

5.6 Email Notifications

- Admin must receive an email for every new inquiry.

- SMTP must from client side (gmail id & Password)

5.7 User Role

- **Super Admin (BarodaWeb Internal)**
 - Includes SEO fields.
 - This user is *not* visible under admin roles.
- **Admin (Client)**
 - Standard access based on permissions.

5.8 Code Optimization

- Remove debugger, console logs, unused code, commented code (except helpful comments).
- Remove unused files before pushing to production. Also check public folders if any unused file while development needs to be removed.
- Optimize all API responses.
- Maintain clean folder structure.

6. Front-End

6.1 Inquiry Forms

- Mandatory fields marked with *.
- Proper validation required.
- After submission: **“Inquiry submitted successfully.”** toast.
- Email notification must be sent to the **user & admin**.

6.2 Sign Up / Sign In

- Mandatory fields with validations.
- Signup triggers email notification to both **admin and user**.

6.3 Forgot Password

Standard Process:

1. The user enters an email.
2. OTP sent to email.
3. The user enters OTP.
4. User sets **New Password + Confirm Password**.
5. Enforce strong password format.

6.4 Design Accuracy

- Must fully match approved reference design / wireframe.
- No layout changes without approval.
- Unauthorized changes will be treated as violations.

6.5 Front-End Form Behavior

- Loader must be shown during submit.
- Success toast shown post-submit.

7. Image, File & Video Standards (Universal SOP)

Feature	Requirement
Max Upload Size	2MB (* Video Upload: as per the project Depended)
Image Format Output	WebP (Mandatory)
Error Handling	Must show descriptive message
Placeholder	“No Image” for missing files
Optimization	Auto resize & crop based on module
File Types	PDF, DOC
Video Format	MP4, M4V

Format Not Allowed	.php.jpg, .php.png, .php.gif, .php.jpeg, .php.webp, .php.svg, .php.pdf, .php.doc, .php.docx, .php.xls, .php.xlsx, .php.ppt, .php.pptx, .php.zip, .php.rar, .php.tar.gz, .php.txt, .php.csv, .php.xml, .php.json, .jsp.jpg, .jsp.png, .jsp.pdf, .asp.jpg, .asp.png, .asp.pdf, .exe.jpg, .exe.png, .exe.pdf, .html.jpg, .html.pdf, .js.jpg, .js.pdf etc... Double extraction or malicious file extraction file not upload in code level
--------------------	---

8. API & Code Quality

8.1 API Development

- Use proper status codes.
- Use validation for every request.
- Apply caching where needed.
- Optimize queries and reduce payload.

8.2 Security

- Use JWT tokens.
- Validate inputs.
- Prevent XSS, CSRF, injections.
- Store secrets in .env only.

8.3 Coding Standards

- Follow ESLint + Prettier formatting.
- Use meaningful variable names. Don't use variables like i, j or single character.
- Use modular code.
- No duplicate logic.

8.4 GitHub Structure

- Every project must maintain at least two core branches: production and development.
- **Developer Responsibilities:**

- Take a pull from the development branch daily before starting work.
- Commit to the development branch daily or at the end of a task.
- Maintain a maximum of two active branches; delete respected branches after merge on development branch older branches upon creating a new one.
- Ensure the team is aware of the project part you are currently working on.
- **Project Leader Responsibilities:**
 - Maintain the production branch.
 - Determine the timing for pushing code to production.
 - Code must be pushed to production only from the development branch.
- **Branch Format :**
 - Maintain the production branch.
 - Determine the timing for pushing code to production.
 - Code must be pushed to production only from the development branch.

More Details : Read page no 14

9. Deployment & Server

- All projects deployed to **BarodaWeb hosting**.
- Use PM2 for backend process management.
- Use environment variables for all credentials.
- Test full website & admin after deployment.
- Share production link only after QA approval.

10. Communication

- Daily end-of-day progress must be shared.
- Report issues immediately if stuck for more than 1 hour.
- Maintain clear and polite communication.

- No implementation without written approval.

11. QA & Testing

11.1 Developer Self-Testing (Mandatory Before Submitting to QA)

Every developer must perform complete self-testing before handing over the project to the QA Team. This includes:

- Full functionality testing of every module.
- End-to-end flow testing (from front-end to admin panel).
- Validation testing for all forms.
- Image, video, and file upload testing.
- Authentication flow testing (Sign-in, Sign-up, OTP, Password Reset).
- Basic security checks (URL access, role protection, input validation).
- Responsiveness testing for all device breakpoints.
- Confirm no console errors or warnings exist.

➡ **Important:** Developers must ensure minimum bugs are passed to QA. Repeated avoidable bugs will be considered SOP violations.

11.2 Cross-Browser Testing (Developer Responsibility)

Every developer must test the project on:

- Google Chrome (Latest Version)
- Mozilla Firefox
- Microsoft Edge
- Safari (If applicable)

All UI, functionality, forms, modals, components, and interactions must work consistently across browsers.

11.3 Cross-Platform Testing (Developer Responsibility)

Testing must be done on:

- Desktop (All major resolutions)
- Tablet
- Mobile (iOS & Android where possible)

11.4 QA Team Testing (After Developer Self-Testing)

Only after successful developer testing, the project should be handed to QA. QA will verify:

- UI/UX accuracy
- Functionality completion
- Edge cases
- Performance
- SEO checks (for Next.js projects)
- Security behavior
- Overall stability**
- Test responsiveness (Mobile, Tablet, Desktop).
- Test all APIs via Postman.
- Validate forms thoroughly.
- Fix all warnings & errors.
- Check Lighthouse performance.
- Check SEO tags.
- Check and inform developers about any unwanted added data which might not need in the project.

12. Warning & Disciplinary Policy

- Every missed SOP point will result in a **formal warning per project**.
- Repeated mistakes will require explanation to **Management & Rupesh Sir**.
- Multiple violations may affect project assignments and performance review.

13. Final Delivery Checklist

- ✓ Design 100% matched
- ✓ All pages functional
- ✓ All API responses validated

- ✓ Images optimized to WebP
- ✓ Mobile responsive
- ✓ Toast & loader implemented
- ✓ Email notification functioning
- ✓ SEO completed (Next.js)
- ✓ No console errors
- ✓ Documentation completed

14. Project Closing Process

14.1. Complete All Scope Tasks

- Finish all assigned modules in the **Scope Module**.
- Update status and add required notes/files.
- No task should remain pending.

14.2. Client Review

- Deploy to staging and get client feedback.
- Complete all final changes.
- Receive written client confirmation.

14.3. Final Deployment

- Deploy on production server.
- Check all pages, forms, and APIs.
- Confirm deployment with the Project Leader.

14.4. Documentation & Backup

- Submit admin guide, API docs, setup guide, and credentials.
- Take code, DB, and media backups.
- Store in approved company storage Server (Bweb Server & GitHub).

14.5. Project360 Closure

- Coordinate with **Project Leader / Team Leader**.
- Upload backups & documentation.
- Review scope completion.
- Project Leader marks status as **Closed** in Project360.
- Collect Google Review from the client side. This responsibility to the Developer and Team Leader.

14.6. Code Cleanup

This not part of project closing, but make your when you work on this project before the code cleanup level, closing time only cross verified

- Remove unused files, console logs, commented code, and test data.
- Clean folder structure and environment files.

14.6. Post-Launch Check (7 Days)

- Monitor errors & performance.
- Fix any urgent issues.

The project is officially closed only after Project Leader updates it in Project360.

WORK PROCESS (Systematic):

1. Create the project in Project360, prepare the project plan, scope, and documentation, and upload the approved scope into Project360.
2. Assign team members based on project requirements and defined responsibilities.
3. Start homepage design based on client preferences; if wireframes are provided, convert them accurately into MERN UI.
4. The development team performs the compression process planning in parallel during UI conversion.
5. Design reusable components and share the initial layout with the client for approval.
6. After layout approval, close the homepage phase and proceed with other common components.
7. Begin layout binding across the application structure.
8. While frontend MERN app development is ongoing, process the admin panel in parallel based on client dynamic requirements.
9. Perform binding logic and make website slots dynamic as per client needs.

10. Conduct continuous basic developer-side QA, including validation checks, data handling, and field accuracy.
11. Implement basic SEO fields during development.
12. Ensure all uploaded images are converted to WEBP format.
13. Share the completed build with the client for review and approval.
14. Before going live, perform project compression by removing unused files, testing CSS, and redundant code.
15. Configure server-side image upload compression.
16. After final approval and checks, deploy the project live.

Note :

This SOP ensures that all MERN/Next.js projects at BarodaWeb meet the highest standards of performance, design accuracy, security, and professionalism. Every developer is required to follow this SOP without exception.

Developer Onboarding & Workflow – Standard Operating Procedure (SOP)

1. Purpose

This document defines the standard operating procedure for developer onboarding and day-to-day development activities. It establishes a controlled, auditable, and secure workflow to ensure code quality, stability, and production readiness.

Objective: Ensure safe and predictable movement of code from local development environments to production through enforced automation and review controls.

2. Environment Setup Requirements

2.1 Prerequisites

The following tools must be installed prior to commencing development:

- Git (Version Control System)
- Node.js (JavaScript Runtime Environment)

- Pre-commit Hooks (installed automatically via `npm install`)
- Visual Studio Code (recommended IDE)

2.2 Initial Setup Procedure

1. Clone the project repository:

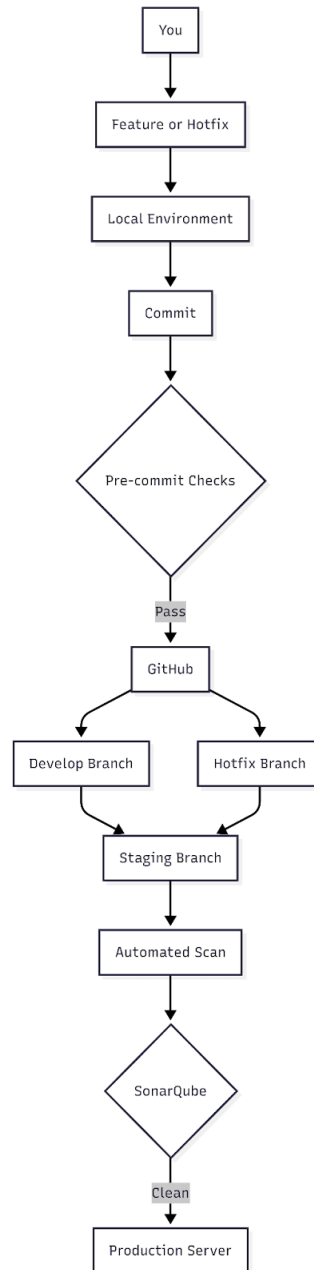
```
git clone <repository-url>  
cd <project-directory>
```

2. Install project dependencies:

```
npm install
```

3. Source Control & Workflow Strategy

3.1 Development Workflow Flowchart



This diagram visually represents the end-to-end development and deployment workflow described in this SOP.

3.2 Branching Model

The project follows a controlled Gitflow-based branching strategy. Code changes must progress through defined stages before release.

3.3 Quality Control Gates

The following enforcement layers are mandatory:

1. **Local Pre-commit Hooks**
 - Automatically executed on `git commit`
 - Enforces formatting (Prettier) and linting rules
2. **Server-side Quality Analysis (SonarQube)**
 - Executed on the staging branch via CI/CD
 - Detects logical defects, security vulnerabilities, and code duplication

4. Development Procedures

4.1 Feature Development (Standard Process)

This process applies to all non-critical development activities.

1. Synchronize with the develop branch and create a feature branch:

```
git checkout develop
git pull origin develop
git checkout -b feature/<TASK NO/ID> - <short-description>
```

2. Implement changes and commit:

```
git add .
git commit -m "feat: <concise description>"
```

3. Push the branch and raise a Pull Request (PR) to `develop`.
 - Minimum one approval is required prior to merge.

4.2 Hotfix Procedure (Production Incidents Only)

This procedure is strictly reserved for production-impacting defects.

1. Create a hotfix branch from `main`:

```
git checkout main
git pull origin main
git checkout -b hot-fix/<issue-description>
```

2. Apply fix and commit:

```
git add .
git commit -m "fix:<TASK NO/ID> - <issue resolution>"
```

3. Raise a PR directly to **main**.
4. Upon deployment, changes are back-merged into **develop**.

5. Naming Conventions & Standards

5.1 Branch Naming

Change Type	Pattern	Example
Feature	feature/<TASK NO/ID> -	feature/tuser-profile
Fix	fix/<TASK NO/ID> -	fix/header-alignment
Hotfix	hot-fix/<TASK NO/ID> -	hot-fix/payment-timeout

I.e :

Example: Feature/MT-02-01/login-page-design

MT-02-01 :- Formula: [Project Code] - [Phase] - [Task No. / Sequence No.]

5.2 Commit Message Standards

All commits must follow semantic conventions:

- feat: New functionality
- fix: Defect resolution
- styles: UI or styling changes
- docs: Documentation updates
- refactor: Non-functional code restructuring

6. Deployment Environments

Environment	Branch	Intended Use
Development	develop	Active development and integration testing
Staging	staging	Quality assurance and pre-release validation
Production	main	Live environment for end users

7. Enforcement & Issue Resolution

7.1 Commit Rejection

Pre-commit failures indicate formatting or linting violations.

Corrective Action: Review terminal output, resolve listed issues, or execute `npm run format`.

7.2 CI/CD Pipeline Failure

Failures during PR validation typically originate from SonarQube findings.

Corrective Action: Review CI logs, address reported code smells or vulnerabilities, and re-run the pipeline.

7.3 Policy Compliance

- Local bypass of hooks (`--no-verify`) is prohibited except under explicit technical approval.
- Server-side checks cannot be bypassed and will block non-compliant merges.

Compliance with this SOP is mandatory for all contributors.